TELEFÓNICA DEL PERÚ — RED DE ACCESO ÓPTICO

UNIVERSIDAD TECNOLÓGICA DEL PERÚ | FACULTAD DE INGENIERÍA DE SISTEMAS

DOCUMENTO TÉCNICO OFICIAL — ENTREGABLE 4.3

ANEXO B: DOCUMENTACIÓN DEL CÓDIGO CON PHPDOC Y JAVADOC

Diccionario de Clases, Contratos DAO, Servicios de Dominio, Patrones de Diseño y Especificación Formal de Métodos

Sistema: Inventario Digital de ODF (ODF Tracker)

Asignatura: Curso Integrador I: Sistemas Software

Docente: Ing. Rudy Chapoñan Camarena

Integrantes: Rodrigo Layme Ureta (U22244493)
Kevin Smith Cusi Espinoza (U23220459)
Diana Greis Aguado Pacompia (U23210141)

Estándar: Javadoc / PHPDoc / PSR-12 / Tipado Estricto PHP 8.1+

Fecha de Emisión: Octubre de 2026

Lima — Perú, 2026

1. Estándar y Convenciones de Documentación PHPDoc / Javadoc

El código fuente del *Sistema de Inventario Digital de ODF* ha sido documentado bajo las directrices internacionales del estándar **PHPDoc / Javadoc**, complementado con tipado de datos estricto (`declare(strict_types=1);`) en la totalidad de clases e interfaces.

> **Convenciones de Bloques Docstring**
> Cada método y clase define formalmente su propósito funcional (Principio SRP), tipos de parámetros obligatorios y opcionales (`@param`), valores de retorno unívocos o nulables (`@return`), excepciones de negocio controladas (`@throws`) y paquetes (`@package`).

La siguiente captura demuestra la integración viva de los bloques de documentación con el entorno de desarrollo integrado (IDE), proporcionando validación estática de tipos, firmas de parámetros y prevención de errores en tiempo de codificación:



```php
namespace App\Entities;

/**
 * Entidad pura de Dominio / DTO: Puerto (conector óptico).
 */
class Puerto
{
    public const ESTADOS_VALIDOS = ['Sin Servicio', 'En Servicio', 'Reservado', 'Malogrado'];

    public function __construct(
        public ?int $idPuerto,
        public int $idOdf,
        public int $posicion,
        public string $estado,
        public ?string $descripcion = null,
        public ?int $idCableConectado = null,
        public ?int $numeroHilo = null,
        public int $bloqueado = 0,
        public ?int $modificadoPor = null,
        public ?string $ultimaModificacion = null
    ) {}

    public static function fromArray(array $data): self
```

Figura B.1: Asistencia Contextual, Validación de Tipos y Documentación Javadoc / PHPDoc en IDE

2. Capa de Contratos e Interfaces DAO (app\\Contracts\\Daos)

Las interfaces DAO constituyen la barrera de desacoplamiento entre la lógica de negocio y el motor relacional MySQL, dando cumplimiento al **Principio de Inversión de Dependencias (DIP)** y permitiendo la ejecución de la suite de 32 pruebas TDD sin requerir una base de datos física activa:

`«interface» PuertoDaoInterface`

Namespace: app\Contracts\Daos\PuertoDaoInterface

Contrato para la consulta, modificación y persistencia atómica de puertos en bandejas de ODF con bloqueo por concurrencia.

Método y Visibilidad	Parámetros (@param)	Retorno (@return)	Excepciones (@throws)
+ PUBLIC `findById(int $id)`	`int $id` : Identificador único del puerto.	?PUERTO Entidad hidratada o null.	`DatabaseException`
+ PUBLIC `findByOdfId(int $odfId, ?int $bandeja)`	`int $odfId` : ID del chasis ODF. `?int $bandeja` : Número opcional de bandeja.	ARRAY<PUERTO> Lista ordenada por posición.	`DatabaseException`
+ PUBLIC `lockForUpdate(int $id)`	`int $id` : Identificador del puerto a bloquear.	?ARRAY Registro bloqueado en modo pesimista.	DATABASEEXCEPTION
+ PUBLIC `updateStatus(...)`	`int $id` , `string $estado` , `?int $cableId` , `?int $fibra` , `?string $obs` , `int $usuarioId` .	BOOL True si se confirmó la transacción.	DATABASEEXCEPTION VALIDATIONEXCEPTION
+ PUBLIC `batchInsert(array $rows)`	`array<array> $rows` : Arreglo de filas validadas.	INT Total de registros insertados.	DATABASEEXCEPTION
+ PUBLIC `obtenerEstadisticasOcupacion(int $odfId)`	`int $odfId` : ID del distribuidor óptico.	ARRAY Conteos de libres, ocupados y saturación.	`DatabaseException`

«interface» OdfDaoInterface & SiteDaoInterface

Namespaces: app\Contracts\Daos\OdfDaoInterface & app\Contracts\Daos\SiteDaoInterface

Contratos para la administración topológica de sedes técnicas (Sites) y bastidores distribuidores (ODF).

Método e Interfaz	Parámetros (@param)	Retorno (@return)	Descripción Funcional
`OdfDao::findById(int $id)`	`int $id` : ID del ODF.	?ODF	Recupera los atributos físicos del chasis (capacidad, rack, sede).
`OdfDao::findBySiteId(int $siteId)`	`int $siteId` : ID del sitio.	ARRAY<ODF>	Lista todos los ODFs desplegados en una central técnica.
`SiteDao::findAllActive()`	Ninguno.	ARRAY<SITE>	Retorna todas las estaciones operativas con conteos de ODFs.
`SiteDao::create(Site $site)`	`Site $site` : Entidad con datos validados.	INT ID generado.	Persiste una nueva central técnica en la base de datos.

3. Capa de Implementaciones DAO Concretas (app\\Daos)

Implementa los contratos DAO utilizando la extensión nativa **PDO**, forzando sentencias preparadas con enlace tipado y desactivando la emulación de consultas (`ATTR_EMULATE_PREPARES => false`) para erradicar cualquier posibilidad de inyección SQL (SQLi).

`class MySqlPuertoDao implements PuertoDaoInterface`

Namespace: app\Daos\MySqlPuertoDao

Implementación transaccional con control de aislamiento ACID y disparo de triggers de auditoría forense.

```php
/**
 * Actualiza el estado físico de un puerto asegurando atomicidad transaccional.
 *
 * @param int $id Identificador del puerto.
 * @param string $estado Nuevo estado operativo (DISPONIBLE, OCUPADO, RESERVADO, DEFECTUOSO).
 * @param int|null $cableId ID del cable de fibra alimentador.
 * @param int|null $fibraNumero Hilo de fibra empalmado.
 * @param string|null $observaciones Justificación técnica del cambio.
 * @param int $usuarioId ID del operador responsable.
 * @return bool True si la operación se completó exitosamente.
 * @throws DatabaseException Si ocurre un fallo en la base de datos o colisión óptica.
 */
public function updateStatus(int $id, string $estado, ?int $cableId, ?int $fibraNumero, ?string $observaciones, in
{
    $this->pdo->beginTransaction();
    try {
        $stmt = $this->pdo->prepare("
            UPDATE PUERTO
            SET estado = :estado, idCableAlimentacion = :cable, numeroHilo = :fibra,
                observaciones = :obs, fechaModificacion = NOW(), modificadoPor = :usr
            WHERE idPuerto = :id
        ");
        $stmt->bindValue(':estado', $estado, PDO::PARAM_STR);
        $stmt->bindValue(':cable', $cableId, $cableId !== null ? PDO::PARAM_INT : PDO::PARAM_NULL);
        $stmt->bindValue(':fibra', $fibraNumero, $fibraNumero !== null ? PDO::PARAM_INT : PDO::PARAM_NULL);
        $stmt->bindValue(':obs', $observaciones, PDO::PARAM_STR);
        $stmt->bindValue(':usr', $usuarioId, PDO::PARAM_INT);
        $stmt->bindValue(':id', $id, PDO::PARAM_INT);
        $stmt->execute();

        $this->pdo->commit();
        return true;
    } catch (PDOException $e) {
        $this->pdo->rollBack();
        throw new DatabaseException("Error transaccional en MySqlPuertoDao: " . $e->getMessage(), (int)$e->getCode
    }
}
```

4. Capa de Servicios de Dominio y Lógica de Negocio (app\\Services)

Encapsula las reglas del negocio de telecomunicaciones de forma desacoplada de controladores y vistas:

`class PuertoBatchImportService`

Namespace: app\Services\PuertoBatchImportService

Servicio de orquestación para la validación y carga masiva de puertos desde archivos CSV/XLSX aplicando el patrón Strategy.

Método	Argumentos	Retorno	Reglas de Negocio Validadas
`importFromFile(...)`	`string $filePath`: Ruta del archivo. `int $odfId`: ID del ODF. `int $usrId`: Operador. `RolePolicyInterface $policy`	**ARRAY** `{ leidos, insertados, errores: [] }`	• Número de puerto dentro del rango (1 a capacidad). • Estado perteneciente al enum válido. • No colisión de hilo óptico en el cable. • Verificación de permisos de rol con la Policy.

`class StatisticsService & class UsuarioService`

Namespaces: app\Services\StatisticsService & app\Services\UsuarioService

Servicios de agregación analítica de infraestructura óptica y autenticación corporativa con seguridad criptográfica.

Servicio / Método	Parámetros	Retorno	Consideraciones de Seguridad
`StatisticsService::calculateStats(...)`	`int $siteId`, `?int $odfId`	**ARRAY** Métricas y semáforos.	Consultas indexadas sin impacto en latencia de producción.
`UsuarioService::authenticate(...)`	`string $correo`, `string $password`	**ARRAY|FALSE** Datos de sesión.	Comparación de contraseña mediante `password_verify()` contra hash BCrypt.
`UsuarioService::changePassword(...)`	`int $id`, `string $oldPass`, `string $newPass`	**BOOL** True en éxito.	Exige contraseña anterior correcta y aplica nuevo hash con salt dinámico.

5. Capa de Patrones de Diseño (Strategy, Factory y Singleton)

Alineación con los estándares SOLID para alta cohesión, desacoplamiento y extensibilidad:

`Patrón Strategy: SpreadsheetReaderInterface, CsvReader, XlsxReader & ReaderFactory`

Namespace: app\Contracts\ReaderInterface & app\Services*

Permite procesar archivos tabulares de forma polimórfica sin que el servicio de importación conozca el formato subyacente.

Clase / Interfaz	Patrón	Firma de Método Principal	Responsabilidad Técnica
`SpreadsheetReaderInterface`	Strategy Interface	`read(string $filePath): array`	Contrato unificado para extracción de filas tabulares.
`CsvReader`	Concrete Strategy	`read(string $filePath): array`	Lectura en streaming con detección de separadores (`,` o `;`) y UTF-8.
`XlsxReader`	Concrete Strategy	`read(string $filePath): array`	Deserialización OpenXML ligera con límites de memoria y mitigación Zip Bomb.
`ReaderFactory`	Simple Factory	`create(string $ext): Reader`	Instancia la estrategia según la extensión MIME del archivo subido.

Patrón Singleton: DatabaseConnection & Componente Csrf

Namespaces: app\Core\DatabaseConnection & app\Core\Csrf

Gestión centralizada del pool de conexiones MySQL y mitigación de falsificación de peticiones en sitios cruzados.

- **DatabaseConnection::getSharedInstance(): PDO:** Retorna una única instancia de PDO compartida durante el ciclo de vida de la petición HTTP, optimizando el consumo de descriptores de socket en el servidor de base de datos.
- **DatabaseConnection::transaction(callable $cb):** Ejecuta una función anónima dentro de un bloque `BEGIN TRANSACTION` con `COMMIT` automático y `ROLLBACK` ante excepciones no controladas.
- **Csrf::token(): string:** Genera un token unívoco pseudoaleatorio mediante el generador criptográfico seguro `random_bytes(32)`.
- **Csrf::validateOrDie(?string $token): void:** Compara el token recibido contra el almacenado en sesión mediante `hash_equals()`, previniendo ataques de tiempo (Timing Attacks).

6. Capa de Políticas de Autorización y Control de Acceso RBAC (app\\Policies)

El sistema implementa el modelo **Role-Based Access Control (RBAC)** a través del patrón **Policy** y la fábrica `PolicyFactory`, garantizando el principio de menor privilegio:

Política / Rol	canModifyPort()	canImportBatch()	canDeleteRecord()	Alcance Operativo
AdminPolicy `(Rol Administrador)`	HABILITADO	HABILITADO	HABILITADO	Control total de infraestructura, usuarios, sedes y auditoría forense.
SupervisorPolicy `(Rol Supervisor)`	HABILITADO (con restricciones)	RESTRINGIDO	RESTRINGIDO	Modificación de notas técnicas, asignación de hilos y exportación de reportes.
TecnicoPolicy `(Rol Técnico Campo)`	HABILITADO (solo libres)	RESTRINGIDO	RESTRINGIDO	Actualización de estado de acopladores en campo. No puede alterar puertos en servicio.

7. Capa de Entidades de Dominio (app\\Entities)

Representan los modelos del dominio de telecomunicaciones con atributos fuertemente encapsulados:

- **Puerto:** Encapsula `idPuerto`, `idOdf`, `posicion`, `estado`, `numeroHilo`, `observaciones`. Implementa `isBloqueado(): bool` para evitar modificaciones concurrentes en empalmes activos.
- **Odf:** Modela el bastidor físico con atributos `idOdf`, `siteId`, `capacidadPuertos` (48/96/144), `ubicacionRack` y código de identificación.
- **Site:** Central técnica o estación repetidora (`idSite`, `nombre`, `departamento`, `distrito`, `tipo`).
- **CableAlimentacion:** Troncal óptica (`idCable`, `siteId`, `capacidadHilos`: 12, 24, 48, 96 fibras).
- **Usuario:** Operador del sistema con control de roles (`idUsuario`, `nombre`, `correo`, `rol`, `estado`).

8. Diagrama de Clases UML del Sistema (ODF Tracker)

La siguiente ilustración técnica consolida la totalidad de clases, interfaces, métodos y relaciones del sistema orientado a objetos:

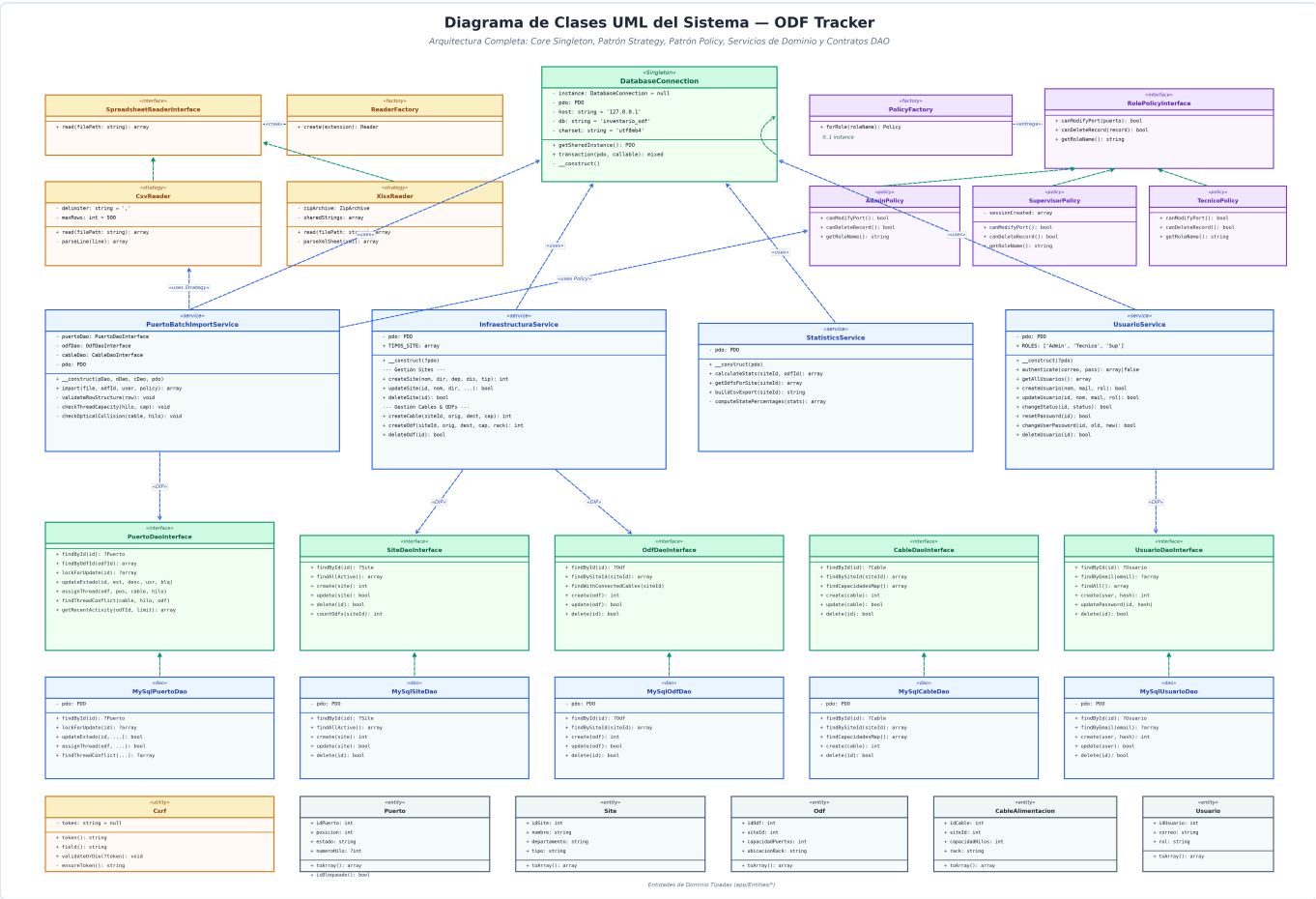


Figura B.2: Diagrama de Clases UML — Arquitectura Completa con Patrones Singleton, Strategy, Policy, Servicios y Contratos DAO